TruffleHog
Praktische Übungen

Aufgabenblatt | Einzelarbeit | Acht Aufgaben für den Einstieg

Verwende ausschließlich die bereitgestellten Testdateien und künstlichen Secrets. Dokumentiere deine Ergebnisse kurz und kürze gefundene Werte ab. Die Befehlsübersicht auf Seite 5 darfst du bei allen Aufgaben verwenden.

Name: _____ Datum: _____

01 Dein erster Scan

10 Minuten

Im ersten Übungsordner befindet sich eine Datei mit einem künstlichen Secret.

1. Öffne das Terminal im Workshop-Verzeichnis. Prüfe mit **trufflehog --version**, ob TruffleHog verfügbar ist.

2. Führe den Scan-Befehl für Aufgabe 1 aus der Befehlsübersicht aus.

3. Suche in der Ausgabe nach Dateipfad, Detektor und erkanntem Wert.

4. Öffne die betroffene Datei und suche die gemeldete Stelle.

Halte fest: In welcher Datei wurde das Secret gefunden? Welcher Detektor hat es erkannt?

02 Ein Projekt durchsuchen

10 Minuten

Der zweite Übungsordner enthält Quellcode, Konfigurationsdateien und ein Backup.

1. Sieh dir die Dateien zunächst ohne TruffleHog an. Notiere, wo du Secrets vermutest.

2. Passe den Ordnerpfad im Scan-Befehl an und scanne das gesamte Anwendungsverzeichnis.

3. Vergleiche die Ergebnisse mit deinen Vermutungen. Prüfe, ob auch Unterordner untersucht wurden.

Halte fest: Erstelle eine Tabelle mit Datei, Detektor und deiner ursprünglichen Vermutung. Warum sollten auch Backups berücksichtigt werden?

Ergebnisse verstehen

Aufgaben 3 und 4

03 Einen Fund richtig verstehen

10 Minuten

Wähle einen Fund aus Aufgabe 2. Nutze die Erläuterungen auf Seite 5.

1. Ermittle Dateipfad und verwendeten Detektor.

2. Prüfe, welche Angaben zur Verifikation angezeigt werden.

3. Lies die Erläuterungen zu Verified, Unverified und Unknown.

4. Erkläre den Unterschied zwischen einem erkannten und einem als gültig bestätigten Secret.

Halte fest: Beschreibe den Fund in drei bis vier Sätzen. Bedeutet ein unbestätigter Fund automatisch, dass der Wert ungefährlich ist?

04 Die JSON-Ausgabe lesen

10 Minuten

JSON ist ein strukturiertes Datenformat zur Weiterverarbeitung durch andere Programme.

1. Wiederhole den Scan aus Aufgabe 2 und ergänze den Parameter **--json**.

2. Suche anhand des Beispiels auf Seite 5 nach Dateipfad, Detektor, erkanntem Wert und Verifikationsangaben.

3. Vergleiche diese Angaben mit der normalen Terminalausgabe.

Halte fest: Übernimm einen JSON-Fund und kennzeichne die gesuchten Angaben. Kürze den Wert ab. Welche Ausgabe eignet sich zum direkten Lesen, welche zur automatischen Weiterverarbeitung? Begründe deine Antwort.

Platz für deine Ergebnisse

Angabe	Beobachtung
Dateipfad	
Detektor	
Verifikationsangaben	

Filter und Git-Historie

Aufgaben 5 und 6

05 Ergebnisse filtern

10 Minuten

Mit Ergebnisfiltern bestimmst du, welche Funde angezeigt werden.

1. Verwende den Scan-Befehl aus Aufgabe 2. Behalte **--no-verification** bei.

2. Führe den Scan mit **--results=unverified** aus.

3. Ersetze diesen Filter durch **--results=verified** und wiederhole den Scan.

4. Vergleiche die beiden Ausgaben.

Halte fest: Notiere beide Befehle und die Unterschiede. Warum erscheinen die künstlichen Secrets bei einem Filter möglicherweise nicht? Beweist eine leere gefilterte Ausgabe, dass keine Secrets vorhanden sind?

06 Das gelöschte Secret

15 Minuten

Im bereitgestellten Git-Repository wurde ein künstliches Secret eingecheckt und später aus der Datei entfernt.

1. Öffne die aktuelle Anwendungsdatei und untersuche ihren Inhalt.

2. Führe einen Dateiscan des aktuellen Anwendungsordners aus.

3. Führe anschließend den Git-Scan aus der Befehlsübersicht aus.

4. Suche in der Ausgabe nach Dateipfad und Commit-Kennung.

5. Zeige den betreffenden Commit mit dem angegebenen Git-Befehl an.

Halte fest: Dokumentiere Dateipfad und Commit-Kennung. Warum findet der Git-Scan einen gelöschten Wert? Reicht es aus, ein echtes veröffentlichtes Secret aus der aktuellen Datei zu löschen?

Regeln anwenden

Aufgaben 7 und 8

07 Eine eigene Erkennungsregel anpassen

15 Minuten

Ein fiktives Unternehmen verwendet Tokens mit dem Präfix CAMPUS-TOKEN-, gefolgt von genau vier Ziffern. Du erhältst eine kommentierte Vorlage und eine Erklärung der Musterzeichen.

1. Ergänze in der Vorlage die markierten Stellen für Präfix und Ziffernanzahl.

2. Scanne die Testdateien mit deiner Konfiguration. Verwende den angegebenen Befehl.

3. Vergleiche passende Werte mit solchen, die ein falsches Präfix oder eine falsche Ziffernanzahl haben.

4. Ändere die Regel auf sechs statt vier Ziffern. Wiederhole den Scan.

Halte fest: Speichere die Konfiguration. Welche Werte wurden vor und nach der Änderung erkannt? Was zeigt das über den Zusammenhang zwischen Regel und Scan-Ergebnis?

08 Ein Projekt selbstständig untersuchen

20 Minuten

Du erhältst ein neues Testprojekt mit aktuellen Dateien und einer Git-Historie. Nutze die Methoden und Hilfestellungen der vorherigen Aufgaben.

1. Scanne die aktuellen Anwendungsdateien und anschließend die Git-Historie.

2. Verwende zusätzlich die bereitgestellte eigene Erkennungsregel.

3. Gib die Ergebnisse eines Scans als JSON aus.

4. Erstelle eine Übersicht mit Datei, Detektor, aktuellem oder historischem Fund und bestätigter oder unbestätigter Gültigkeit.

Halte fest: Beantworte: Was zeigt ein Dateiscan? Was ergänzt der Git-Scan? Was unterscheidet Erkennung und Verifikation? Warum bedeutet „kein Treffer“ nicht automatisch „sicher“?

Befehlsübersicht

Alle Befehle im Wurzelverzeichnis des Workshop-Pakets ausführen.

Befehle für Git Bash unter Windows sowie macOS/Linux. TruffleHog muss im Suchpfad verfügbar sein.

Aufgaben 1 bis 5: Dateien scannen

```
trufflehog filesystem aufgabe1/app \
  --config workshop.yml --no-verification --no-update
```

Für Aufgabe 2 ersetze aufgabe1/app durch aufgabe2/app. Für Aufgabe 4 ergänze --json. Für Aufgabe 5 ergänze jeweils genau einen Ergebnisfilter.

Aufgabe 6: Aktueller Stand und Historie

```
trufflehog filesystem aufgabe6/app \
  --config workshop.yml --no-verification --no-update

trufflehog git file://aufgabe6 \
  --config workshop.yml --no-verification --no-update

git -C aufgabe6 show <Commit-Kennung>
```

Ersetze <Commit-Kennung> durch die angezeigte Kennung, ohne die spitzen Klammern.

Aufgabe 7: Eigene Konfiguration

```
trufflehog filesystem aufgabe7/app \
  --config aufgabe7/meine-regel.yml \
  --no-verification --no-update
```

Für Aufgabe 8 verwende aufgabe8/app beziehungsweise file://aufgabe8. Die zusätzliche Regel liegt in aufgabe8/erweiterte-regeln.yml.

Status und JSON-Felder

Angabe	Bedeutung
Verified	Gültigkeit wurde bestätigt.
Unverified	Erkannt, aber nicht als gültig bestätigt.
Unknown	Eine versuchte Überprüfung scheiterte an einem Fehler.
DetectorName / Raw / Verified	Detektor / erkannter Wert / Bestätigung der Gültigkeit.
SourceMetadata	Angaben zur Quelle, zum Beispiel Dateipfad; bei Git auch Commit.

Ein schematischer Fund kann z. B. "DetectorName": "…", "Raw": "…" und "Verified": false enthalten. Die genaue Struktur hängt von Quelle und Version ab. Verified: false allein unterscheidet nicht alle Ursachen fehlender Bestätigung.